Does Every Window Need Its Own Editing Strength?

Support metrics, support-based μ and α, learned settings, and what we are trying now

Karan Kishore

IIT Madras

Weekly Meet | October 28, 2026

Roadmap

1. **Setting**: the data, the score, and what GTA does to the data.
2. **Three ways to measure support** (how well-covered a window is), and why we trust them.
3. **Choosing the editing strength μ from support**: effect on the trained policy.
4. **Choosing the reward push α**: why the score rises and then falls.
5. **Letting an optimiser (CMA-ES) choose μ and α** for each window.
6. **What the support groups actually contain**, and why support did not help.
7. **Looking at the windows** in the simulator.
8. **The current idea**: repairing whole edited windows.

The setting: learning from a fixed dataset

Offline reinforcement learning: a policy is trained only from recorded data; it never interacts with the environment while learning.

Environment: HalfCheetah, a simulated two-legged cheetah. The reward at each step is essentially its forward speed.

Dataset: `halfcheetah-medium-v2`, 1,000 runs of 1,000 steps each, recorded by one medium-skill policy that runs at about 5 m/s. At every step we have the *state* (17 numbers: body angles and velocities), the *action* (6 motor forces) and the reward.

Score: we train TD3+BC (a standard offline RL algorithm) and run the trained policy. Its total reward is rescaled so that

a random policy scores **0** and an expert policy (about 12 m/s) scores **100**.

Training on the dataset alone gives **48.7**.

What GTA does: edit short windows to earn more reward

A **window** is 31 consecutive steps of one run (states, actions, rewards) plus the final next state.

GTA (Lee et al., NeurIPS 2024) trains a *diffusion model* on all windows. A diffusion model learns to turn random noise back into realistic windows.

To **edit** a window, GTA

1. adds a chosen amount of noise to it,
2. lets the model remove the noise while asking for a higher total reward,
3. adds the edited window to the training data.

Two settings control an edit:

μ**, editing strength (0 to 1):** how much noise is added, i.e. how much of the window is rewritten. Small μ: a light touch-up. GTA uses $\mu = 0.5$.

α**, reward push:** the edit asks for $\alpha\times$ the window's original return. $\alpha = 1$: no push. GTA uses $\alpha = 1.3$.

GTA uses the **same** μ and α for every window. With them, the score rises from 48.7 to about **58.5**.

The question

Hypothesis: the best editing strength μ differs from window to window, and depends on two properties of the window:

- **Support:** how many similar windows the dataset contains. Expectation: the model knows well-covered regions, so those windows can be edited *more strongly* without becoming unrealistic.
- **Quality:** how much reward the window itself earns. Expectation: a window that is already good has less to gain, so it should be edited *less*.
- **Downstream:** choosing μ per window should raise the TD3+BC score.

To test this we need a number for the quality and a number for the support of every window.

Physics error, used throughout: we replay an edited window in the real simulator and measure how far each step's claimed next state is from the state the simulator actually produces. An edit counts as **broken** when this error is larger than the average error reported in the GTA paper (0.0085).

Quality: the reward a window earns

$$q_i = \sum_{t=0}^{30} 0.99^{\,t}\, r_{b_i+t}$$

- r_{b_i+t}: the reward at step t of window i, which starts at step b_i of its run.
- 0.99^t gives slightly less weight to later steps (the usual discount in RL).
- Every metric is turned into a **percentile**: the fraction of all windows with a lower value. A window with quality percentile 0.9 earns more than 90% of windows.

Typical values (20,000 random windows): 10% of windows earn less than 102, half earn less than 134, 90% earn less than 152.

Why only the reward inside the window? The reward still to come until the end of the run shrinks for windows late in a run even if the running is equally good, so it would describe *position* in the run rather than behaviour.

Support: three ways to measure how familiar a window is

A window has **high support** when the dataset contains many similar situations (similar postures, speeds and motor forces), and **low support** when it is unusual, for example the first steps of a run when the cheetah speeds up from rest.

Support cannot be observed directly, so we estimated it in three independent ways:

name	what it asks	cost
S_1	**the data:** how far away are the 20 most similar moments from other runs?	fast
S_2	**the model:** how well can the diffusion model repair the window after we add noise?	fast
S_3	**the model:** how probable does the model consider the window, computed exactly?	very slow

S_2 is the one we used for all experiments. S_1 and S_3 are there to check it.

S_1: distance to similar moments in other runs

1. Put all 23 numbers of a step (17 state, 6 action) on a common scale: subtract each number's dataset average and divide by its spread (standard deviation). A difference of 1 then means "one usual spread" for every quantity.
2. For each of the 31 steps of the window, find the **20th most similar step** among the 1,000,000 steps of *other* runs and record its distance:

$$d = \sqrt{\sum_{k=1}^{23} (z_k - z'_k)^2} \quad \text{(straight-line distance over the 23 numbers)}$$

3. The window's value is the middle value (median) of these 31 distances.

Reading it: a short distance means many runs pass through similar situations (crowded region, high support); a long distance means the window is unusual.

Why other runs only: neighbouring steps of the same run are nearly identical, so they would always be the closest and say nothing about coverage.

Typical values: half of the windows have a distance below 1.10; 90% are below 1.49; the most unusual reach 10.

S_2: how well the model repairs a noisy window

1. Take the window (all 32 rows $\times$ 24 numbers, on the common scale).
2. Add random noise of strength σ. We use three strengths, $\sigma = 0.16,\ 0.5,\ 2.4$: exactly the amounts of noise GTA adds when it edits with $\mu \approx 0.25,\ 0.35,\ 0.5$.
3. Ask the diffusion model to remove the noise in one step, without asking for any particular reward.
4. Measure the **repair error**: the average squared difference between the repaired window and the original, over all its numbers.
5. Repeat 4 times with fresh noise and average, so one lucky or unlucky draw does not decide.
6. Combine the three noise strengths: for each strength, compare the window's error with all other windows' errors; average the three comparisons. **Smaller error $\Rightarrow$ higher S_2.**

Typical errors (median window): 0.0045 at $\sigma = 0.16$, 0.022 at $\sigma = 0.5$, 0.15 at $\sigma = 2.4$.

Why the repair error measures support

Analogy. Show a person a smudged photo of a common street sign: they read it correctly. Show them a smudged photo of a sign they have never seen: they guess, and usually guess wrong. How well someone can un-smudge a picture tells you how familiar that kind of picture is to them.

For the diffusion model:

- It was trained to remove noise from the windows of this dataset.
- For a kind of window it has seen many times, it removes the noise accurately: **small error, high support**.
- For a rare kind of window, it "repairs" it towards the common windows it knows: **large error, low support**.

Link to probability: it is a known property of diffusion models that windows the model considers more probable are repaired with smaller error. So S_2 is a cheap stand-in for "how probable does the model find this window". The exact version of that question is S_3 (next slide).

Why these three noise strengths: they are the noise levels an actual edit uses, so S_2 asks "can the model handle this window under the noise it will get when we edit it?"

S_3: the exact probability the model gives a window

- S_3 is the logarithm of the probability density the trained diffusion model assigns to the window: **higher means the model finds the window more likely**.
- **How it is computed, in words:** the window is moved, smoothly and without any randomness, from its clean form to pure noise along a path fixed by the model. Where it ends up in the noise has a known probability. Along the way we keep track of how much the space around the window is stretched or squeezed. Combining "where it ends up" with "how much it was stretched" gives the window's exact probability under the model.
- The stretching is tracked with a standard random shortcut (Hutchinson's trick) so it costs one extra network pass per step instead of 768.
- It takes minutes per window, so we computed it for **100 random windows** only.

Values: between 605 and 2,238 (log scale). The numbers are large because a window has 768 numbers and their contributions add up in the log; only the *order* of windows matters.

Role: S_3 is exactly what S_2 approximates, so it is the natural check for S_2. It is the model's view, not the absolute truth about the data.

The three support metrics side by side

	S_1 distance to similar moments	S_2 repair error	S_3 exact probability
looks at	each step separately	the whole window	the whole window
uses the model	no	yes	yes
cost for 20,000 windows	seconds	about 3 seconds	only 100 windows feasible
main strength	direct, no model needed	tests the model at the noise editing uses	exact for the model
main weakness	ignores the order of steps; every number weighs equally	depends on the model; 3 noise levels chosen by hand	very slow; one random shortcut per window
role	cross-check	**used in all experiments**	check for S_2

How we check that the metrics mean something

There is no ground truth for support, so we check three things:

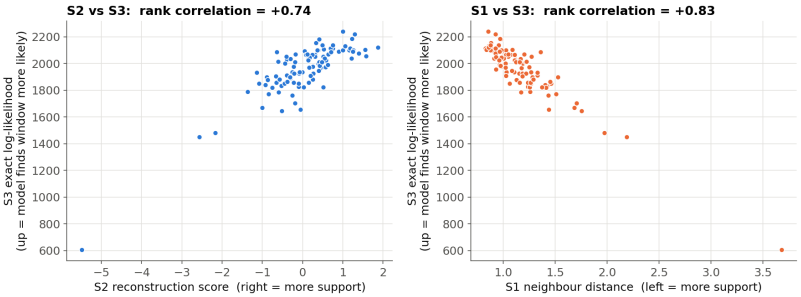
1. **Agreement:** do the three metrics, built on different ideas, put windows in a similar order?
2. **Not just quality:** is support different from simply "earns a lot of reward"?
3. **Prediction:** does support predict which edits break physics, as the hypothesis says it should?

The tool: rank correlation (Spearman's ρ). Sort the windows by metric A and by metric B, and compare the two orderings.

- $\rho = +1$: identical order. $\rho = 0$: unrelated. $\rho = -1$: exactly reversed.
- Rough guide: around ± 0.3 a weak but real tendency; around ± 0.8 strongly related.
- Only the order is compared, so it does not matter that the metrics have different units.

Each ρ comes with a 95% range: we re-draw the windows at random 5,000 times, recompute ρ, and keep the middle 95% of the results.

Agreement: S_2 and S_1 against the exact probability S_3



Each dot is one of the 100 windows for which S_3 was computed. **Vertical axis:** S_3, higher = the model finds the window more likely. **Left, horizontal:** the S_2 score, further right = smaller repair error. **Right, horizontal:** the S_1 distance, further left = more similar moments nearby.

- Left: dots rise from left to right, $\rho = 0.74$ (95% range 0.62 to 0.82). Right: dots fall from left to right, $\rho = 0.83$.
- The single window at the bottom (605 on S_3) is extreme on all three metrics; without it the correlations barely change (0.73 and 0.82).

Is S_2 just noisy? Re-measuring it more carefully

Question: S_2 uses only 4 random noise draws per strength. Is that why it is not closer to S_3?

Test: recompute S_2 on the same 100 windows four more times, each with 4 fresh draws, and also with all 16 draws together.

version of S_2	rank correlation with S_3
original (4 draws)	0.74
four new sets of 4 draws	0.76, 0.76, 0.76, 0.74
16 draws	**0.80** (95% range 0.69 to 0.87)
only the weakest noise, $\sigma = 0.16$	0.75
only the middle noise, $\sigma = 0.5$	0.71
only the strongest noise, $\sigma = 2.4$	0.66

- Two independent 4-draw versions order windows almost the same way ($\rho = 0.88$ with each other), so the randomness is small. More draws close only part of the gap (0.74 $\rightarrow$ 0.80).
- The rest is by design: S_2 looks only at the noise levels editing uses, while the exact probability depends most on very fine detail (the weakest noise tracks S_3 best).

Support is not quality, and it predicts broken edits

Not just quality: rank correlation with quality is 0.42 for S_2 and 0.49 for S_1. Unusual windows tend to earn a bit less, but the two are far from the same, so they can be studied separately.

Prediction: 400 windows were edited at 7 strengths and every edit was replayed in the simulator. Rank correlation between a window's metric and the physics error of its edit:

editing strength μ	0.1	0.2	0.3	0.35	0.4	0.45	0.5
S_2 (support)	-0.29	-0.36	-0.34	-0.28	-0.24	-0.18	-0.13
S_1 (support)	-0.25	-0.34	-0.32	-0.26	-0.28	-0.25	-0.15
quality	$+0.07$	$+0.01$	$+0.01$	$+0.09$	$+0.04$	-0.14	-0.07

- Negative = the more support, the *smaller* the physics error. Both support metrics show this at every μ; quality does not.
- The effect is moderate and **fades towards** $\mu = 0.5$: with that much noise the model rewrites almost the whole window, so the original window matters less.

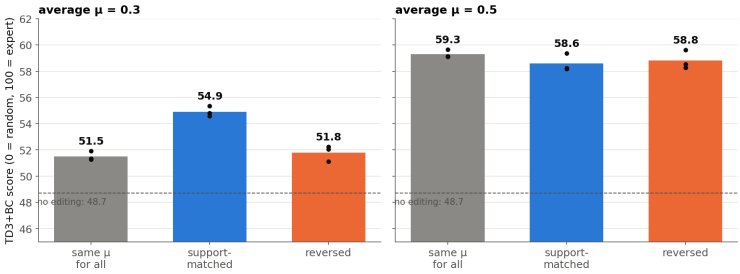
Choosing μ from support: the experiment

- **Same windows everywhere:** 161,291 windows (about 5 million edited steps). Only the μ given to each window changes between versions.
- Windows are split into **5 equal groups by** S_2 (quintiles), from least to most support.
- **Support-matched:** more support gets a stronger edit. **Reversed:** the same values in the opposite order. **Same for all:** one μ with the same average. The reversed version checks that any gain comes from the *matching*, not just from mixing different μ values.

	version	group 1 (least)	group 2	group 3	group 4	group 5 (most)	average
gentle	same for all	0.30	0.30	0.30	0.30	0.30	0.3
	support-matched	0.20	0.25	0.30	0.35	0.40	0.3
	reversed	0.40	0.35	0.30	0.25	0.20	0.3
GTA-level	same for all	0.50	0.50	0.50	0.50	0.50	0.5
	support-matched	0.30	0.40	0.50	0.60	0.70	0.5
	reversed	0.70	0.60	0.50	0.40	0.30	0.5

The gentle support-matched values follow the measured "safe μ" per group (the strongest edit before physics breaks: 0.20 for the least supported windows up to 0.34 for the most supported). TD3+BC: 1 million training steps, 3 random seeds per version.

Choosing μ from support: effect on the trained policy



Vertical axis: TD3+BC score. **Bars:** average over 3 seeds; **black dots:** the individual seeds. **Dashed line:** training without any edited data (48.7).

- **Gentle editing (left):** support-matched beats same-for-all by **+3.4** and reversed by **+3.1**. The seeds do not overlap, and the result repeated in a second, independent implementation ($+2.5$ and $+1.8$).
- **GTA-level editing (right):** support-matched is **not** better (-0.7 against same-for-all, within seed noise).
- The best version overall is simply $\mu = 0.5$ for everyone (59.3). Support-based μ helps only in the gentle regime, which scores lower anyway.

Choosing the reward push α: the experiment

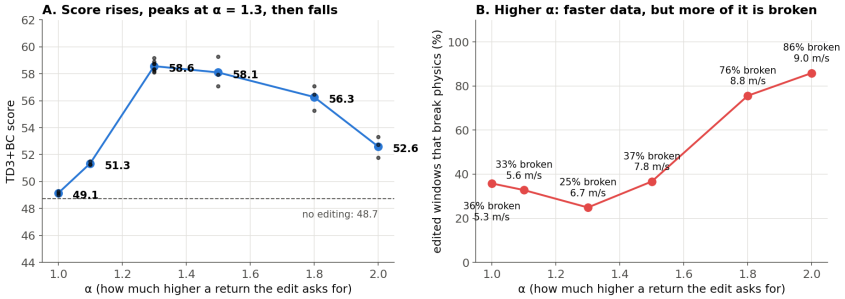
μ is fixed at GTA's 0.5 and the reward push α is varied. 32,258 windows (1 million edited steps), TD3+BC as before.

1. **One α for everyone:** 1.0, 1.1, 1.3, 1.5, 1.8, 2.0.
2. α **from support**, around the best value 1.3, using the same 5 support groups:

version	group 1 (least)	group 2	group 3	group 4	group 5 (most)
up	1.1	1.2	1.3	1.4	1.5
down	1.5	1.4	1.3	1.2	1.1
random	each window gets one of 1.1–1.5 at random				

How to read the second part: if "up" beats "random", matching α to support matters. If "up" and "random" do equally well, only the *mixing* of several α values matters.

More reward push helps, up to a point



A: horizontal = α; vertical = TD3+BC score; big dots = average, small dots = seeds; dashed = no edited data. **B:** same horizontal axis; vertical = share of edited windows whose physics error is above the paper's average; labels also give the edited data's average speed.

- From $\alpha = 1$ to 1.3 the edited data gets faster (5.3 $\rightarrow$ 6.7 m/s) while staying about as realistic, and the score climbs from 49.1 to **58.6**.
- Beyond 1.3 the data keeps getting faster (up to 9 m/s), but the share of broken windows jumps (37% $\rightarrow$ 76% $\rightarrow$ 86%) and the score falls to 52.6.

Why the score falls: fast but impossible motions

- **Reward is speed**, so faster edited data shows the policy how to earn more. The dataset itself never runs faster than about 7.3 m/s; GTA's edits at $\alpha = 1.3$ add running that is faster than almost anything in the data.
- **But the model has never seen very fast running.** Asked for much more reward ($\alpha = 1.8$–2), it produces windows that look fast but cannot happen physically: in one rendered example the cheetah stands almost vertically on its back legs while supposedly moving at 10 m/s.
- **TD3+BC learns from every step it is given.** Impossible steps teach it that certain actions lead to fast, high-reward states that the real simulator never reaches. The policy chases those, and the real score drops.

The trade-off in one line: more speed in the data helps; broken physics hurts. $\alpha = 1.3$ is where the two balance.

Choosing α from support: results

version	score (mean $\pm$ spread)	seeds	compared with $\alpha = 1.3$ for all
$\alpha = 1.3$ for all	58.57 $\pm$ 0.38	8	–
up (more push for more support)	59.53 $\pm$ 0.51	8	+0.96
down (less push for more support)	59.18 $\pm$ 0.62	8	+0.61
repeated with 5 new seeds, including the random control			
$\alpha = 1.3$ for all	58.66 $\pm$ 0.44	5	–
up	59.52 $\pm$ 0.58	5	+0.87
down	59.40 $\pm$ 0.58	5	+0.74
random	58.69 $\pm$ 1.72	5	+0.04

- "Up" is reliably about +0.9 above a single α, on both sets of seeds.
- But "down" (the opposite direction) does nearly as well, and "up" cannot be told apart from "random" (the random version varies a lot between seeds).
- **Conclusion:** mixing α values may help slightly; **matching α to support does not clearly matter**.

Letting an optimiser choose μ and α per window

Instead of hand-made rules, write μ and α as **smooth curves** of a window's support S and quality Q (both percentiles, shifted to run from -0.5 to $+0.5$):

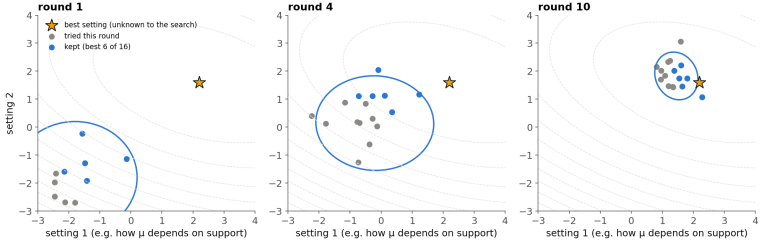
$$\mu_i = 0.3 + 0.4 \cdot \mathrm{sig}(a_0 + a_S S_i + a_Q Q_i), \qquad \alpha_i = 1.0 + 0.8 \cdot \mathrm{sig}(c_0 + c_S S_i + c_Q Q_i)$$

- $\mathrm{sig}(z) = 1/(1+e^{-z})$ squeezes any number into the range 0 to 1, so μ always stays between 0.3 and 0.7 and α between 1.0 and 1.8.
- The six numbers $a_0, a_S, a_Q, c_0, c_S, c_Q$ decide the rule. Example: $a_S > 0$ means more support gets a stronger edit; $a_S = a_Q = c_S = c_Q = 0$ means one μ and one α for everyone.

Goal: find the six numbers that give the highest TD3+BC score.

Difficulty: the score is only known after generating edited data and training a policy (minutes per try), and there is no formula telling us which direction improves it. We need a search method that only needs "try a setting, observe its score". We used **CMA-ES**.

CMA-ES explained: guided trial and error



A toy problem with two settings (horizontal and vertical axes). Grey contour lines mark equal score; the star is the best setting, which the search does not know. Each round:

1. **Try:** draw 16 settings at random from a cloud (the blue ellipse marks where the cloud is). Measure each one's score.
2. **Keep the best:** take the best 6 (blue dots); drop the rest (grey).
3. **Move and reshape the cloud:** new centre = average of the kept settings. The cloud is stretched along the direction in which the kept settings lie and in which the centre just moved, and shrinks when progress slows.

Round 1: the cloud is far away. Round 4: it has moved towards better scores. Round 10: it has shrunk around the star.

CMA-ES in a little more detail

The name: Covariance Matrix Adaptation Evolution Strategy.

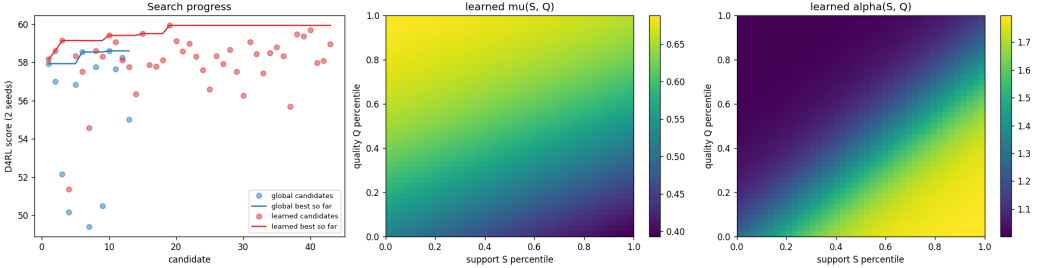
- **Evolution strategy:** like breeding. Each round produces a "population" of candidate settings; the fittest survive and the next population is drawn around them.
- **Covariance matrix:** the shape of the cloud (how wide it is in each direction, and how tilted). "Adaptation" means this shape is learned from which candidates did well.

What one round does, precisely:

1. Draw candidates $x_k = m + s \cdot$ (random step shaped by the cloud), where m is the centre and s the overall step size.
2. Score each x_k (here: generate data, train TD3+BC, read the score).
3. New centre $m =$ weighted average of the best candidates (better ones count more).
4. Update the cloud shape towards the directions of the successful steps, and remember the recent direction of travel so consistent progress stretches the cloud further.
5. Make s larger if the centre keeps moving in the same direction (far from the goal), smaller if it jumps back and forth (close to the goal).

Why it suits us: it needs only scores, tolerates noisy scores, and works well with a handful of settings (here 6).

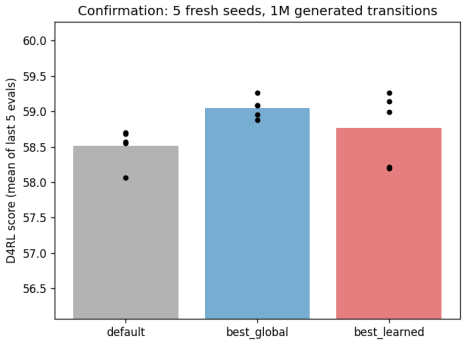
The search, and what it learned



Left: horizontal = candidate number in the order tried; vertical = score (2 seeds, half-size data). Blue = searching one μ and one α for everyone (13 tries); red = searching the full per-window rule (43 tries); lines = best so far. **Middle / right:** the best per-window rule found. Horizontal = support percentile, vertical = quality percentile, colour = μ (middle) and α (right) that the rule gives such a window.

- The learned rule gives *lower* μ to well-supported windows and pushes α hard only for well-supported, low-quality windows: the opposite of our hypothesis for μ.

Confirming the winners on fresh seeds

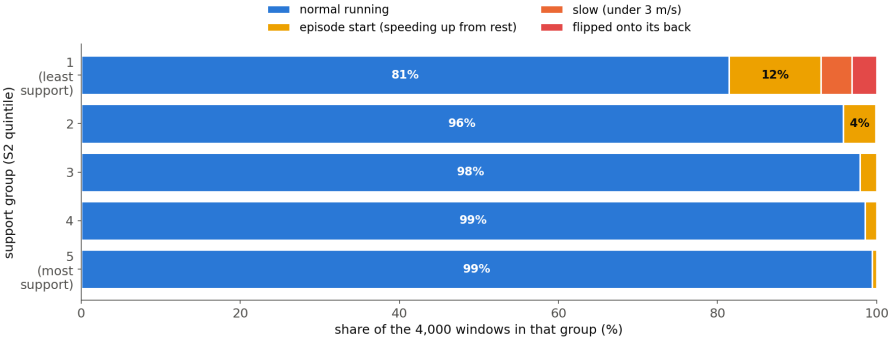


Vertical = score (note: the axis starts at 56); bars = average of 5 new seeds; dots = individual seeds.

setting	score	avg. μ	avg. α
GTA default	58.51	0.500	1.300
best single μ, α	**59.05**	0.565	1.356
best per-window rule	58.76	0.592	1.277

- The per-window rule is **not** better than one well-chosen μ and α (-0.29, within seed noise).
- During the search it looked better (59.9), but that came from luck: when 43 settings are each tried on only 2 seeds, one of them looks good by chance.
- A slightly stronger single setting (0.565, 1.356) gives a small real gain over GTA's default: $+0.54$.

What the support groups actually contain



20,000 windows, split into 5 equal groups by S_2 (rows, least support at the top). Each bar shows what kinds of windows the group contains; horizontal axis = share of the group. Categories come from the cheetah's body angle, speed and position in its run.

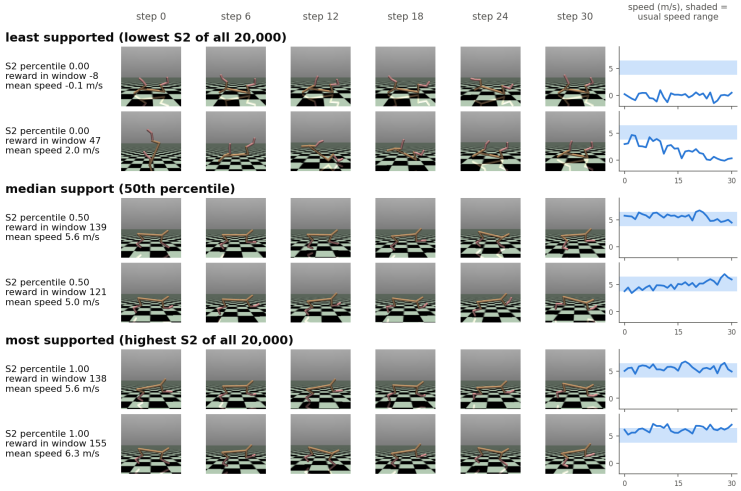
- Even the least-supported group is **81% normal running**. The truly unusual windows (flips, slow-downs, starts from rest) are only about 5% of all data, and they all sit in group 1.
- Groups 2 to 5 are 96–99% normal running and differ only in small details.

Why support-based μ and α could not do much here

- **One policy, one way of running.** The dataset was recorded by a single medium-skill policy, so about 95% of all windows are the same kind of steady running. Support differences between most windows are small.
- **The unusual windows are few and not useful to edit.** A cheetah lying on its back gains nothing from a stronger or weaker edit; these windows are a small part of the data either way.
- **At GTA's own setting the original window matters little.** With $\mu = 0.5$ the model rewrites most of the window, so how familiar the original was hardly changes the result (the link between support and physics error fades towards $\mu = 0.5$).
- **Support describes the input, but RL is affected by the output.** Whether an edited window is realistic is a property of the edited window; the original's support predicts it only weakly.

Summary: the support metric works (it finds the unusual windows), but on this dataset there is too little variation in support among the windows that matter for it to steer μ or α usefully.

What low, medium and high support look like



Two windows at each level, rendered in the simulator at 6 moments of the 31 steps. Right: forward speed over the window (blue line) and the dataset's usual speed range (shaded).

Reading the renders

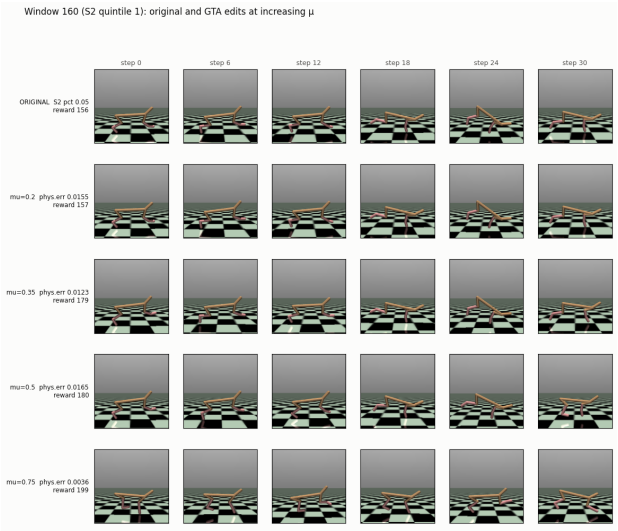
- **Least supported:** the cheetah has **flipped onto its back** with its legs in the air (first window), or is rolling over (second window, upright at step 0, on its back by step 12). Speed falls to about 0, far below the usual range, and the window earns almost no reward.
- **Median support:** normal running; speed stays inside the usual range with small ups and downs.
- **Most supported:** the cleanest, most regular stride; the torso stays level and the legs repeat the same cycle every few steps.

In numbers, over 20,000 windows:

- 94.6% normal running, 4.0% starts from rest, 0.8% slow, 0.6% flipped.
- **All** 124 flipped windows, 97% of the slow windows and 59% of the starts fall in the least-supported fifth.

So S_2 does what it should: it ranks stumbles and flips as unusual and steady running as familiar. The middle of the ranking separates only slightly irregular from slightly more regular running.

Editing an unusual window with increasing μ

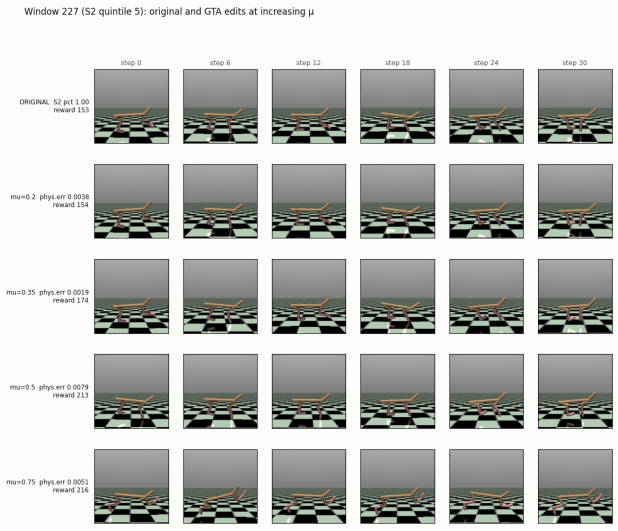


A stumbling window (least-supported fifth): the body pitches nose-down at steps 18–24 and speed drops.

Rows: the original, then GTA edits with $\mu = 0.2, 0.35, 0.5, 0.75$; columns: 6 moments of the window. Left labels: physics error and true reward.

- $\mu = 0.2$–0.5: the edit keeps the stumble but pushes it towards more reward (156 $\rightarrow$ 180). Physics error is high (0.012–0.017, above the 0.0085 limit): the model forces an unusual motion it does not know well.
- $\mu = 0.75$: the stumble disappears; the model replaces it with a normal stride (error 0.0036, reward 199). It is no longer the original window.

Editing a familiar window with increasing μ

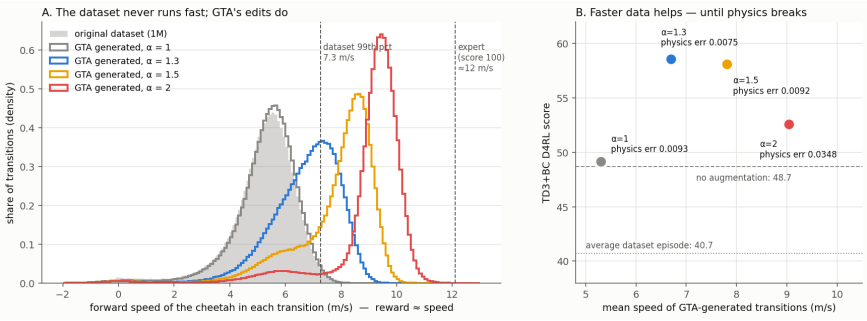


A steady running window (most-supported fifth), same layout as before.

- $\mu = 0.2$–0.35: almost the same motion, a bit faster (reward 153 $\rightarrow$ 174) with very low physics error (0.002–0.004).
- $\mu = 0.5$: reward jumps to 213 (+39%) while the error (0.0079) stays below the limit.
- $\mu = 0.75$: the posture visibly changes into a longer, stretched stride.

Together with the previous slide: familiar motions can be edited much more strongly before physics breaks; unusual ones break even under light edits. This is the support effect we measured, now visible. It is real, but it concerns few windows on this dataset.

What limits the score



A: horizontal = forward speed of each step; vertical = how common that speed is. Grey = the dataset; coloured lines = GTA's edited data for different α. Dashed lines: the dataset's top 1% speed (7.3 m/s) and the expert's speed (12 m/s). **B:** horizontal = average speed of the edited data; vertical = TD3+BC score; labels give the data's physics error.

- The policy can only learn speeds the data shows. GTA helps by adding faster running; the limit is that **faster edits break physics**.

What every attempt so far got right, and wrong

For TD3+BC, edited data has to be three things **at the same time**:

1. **fast:** new, faster running than the dataset has;
2. **realistic:** each step follows the real physics;
3. **connected:** each step's next state is exactly the next step's starting state, so the 31 steps form one continuous motion. TD3+BC learns how valuable a state is by passing value backwards along these links (value here = reward now + value of where I end up).

approach	fast	realistic	connected
GTA with strong push ($\alpha = 1.8$–2)	**yes**	**no**	**yes**
correct each step separately with a physics model	**yes**	**yes**	**no**
re-simulate the whole window with a physics model	**no**	**no**	**yes**
whole-window adjustment (current idea)	**yes**	**yes**	**yes**

Correcting each step separately made the data more realistic but lowered the score by about 2 points: the broken links stopped value from flowing back along the fast running.

The current idea: adjust the whole window at once

Tool: a small *physics model* trained on the dataset only. Given a state and an action, it predicts the next state. It is accurate for **one** step but its small errors pile up over many steps. It is really 5 small networks; where they disagree, the model is unsure.

Method: treat GTA's edited window as a draft and nudge *every* state and *every* action a little, all together, to satisfy three wishes at once:

1. **stay close to GTA's draft**, so its faster motion is kept;
2. **every single step obeys the physics model**: predicted next state $\approx$ the window's next state (only one-step predictions, where the model is accurate);
3. **stay where the 5 networks agree**, so the adjustment cannot exploit places where the model is wrong.

$$\text{minimise } \underbrace{\|\hat{s} - s_{\text{GTA}}\|^2}_{\text{close to draft}} + 0.1 \underbrace{\|\hat{a} - a_{\text{GTA}}\|^2}_{\text{similar actions}} + 100 \underbrace{\textstyle\sum_t \|\hat{s}_{t+1} - f(\hat{s}_t, \hat{a}_t)\|^2}_{\text{each step obeys physics}} + 10 \cdot \underbrace{\text{disagreement}}_{\text{model is sure}}$$

Repeated 500 times: "move every state and action slightly in the direction that lowers this sum". Each adjusted state is both the end of one step and the start of the next, so the window **stays connected automatically**.

Why we expect it to help, and first checks

Reasoning: the score is limited by how much *fast and valid* running TD3+BC sees; strong pushes failed because their data broke physics; fixing physics step by step failed because it broke the links. The adjusted data is fast, realistic and connected.

First check, before any RL (300 windows each, replayed in the real simulator):

data	speed	physics error	broken windows
GTA default ($\alpha = 1.3$)	6.7 m/s	0.0075	25%
GTA, $\alpha = 1.5$, as generated	7.9 m/s	0.0093	36%
GTA, $\alpha = 1.5$, **adjusted**	**7.8 m/s**	**0.0049**	**6%**
GTA, $\alpha = 1.8$, as generated	8.8 m/s	0.0190	77%
GTA, $\alpha = 1.8$, **adjusted**	**8.7 m/s**	**0.0071**	**25%**

- The adjustment moves states by only about a tenth of their usual spread, keeps the speed, and removes most of the broken physics.
- **Limits:** realism is enforced by the learned physics model, not the real simulator, so it is only as good as that model. Whether this raises the TD3+BC score is being tested now (3 seeds each for $\alpha = 1.5, 1.8, 2$).

Summary

- **Support metrics work:** three independent measures order windows similarly ($\rho = 0.74$–0.83 against the exact probability), differ from quality, and predict which edits break physics. The renders confirm it: lowest support = flipped cheetah, highest = cleanest stride.
- **Support-based μ** helps only with gentle edits ($+3.4$); at GTA's strength one μ for everyone is as good.
- **α:** more reward push helps until the edited data breaks physics; $\alpha = 1.3$ balances speed and realism. Matching α to support is no better than mixing α at random.
- **CMA-ES learned rule** is no better than one well-chosen setting; a slightly stronger single setting gives $+0.5$.
- **Why:** 95% of this dataset is the same kind of running, so support varies little where it matters; and the realism of the *edited* window, not the original's support, is what affects RL.
- **Now:** adjusting whole edited windows so they are fast, realistic and connected at the same time. Data checks look promising; RL results are coming in.

Questions?